

Exercício Prático: API de Reservas e Avaliações do Imóvel Lions

Turma: LionsDev

Tópicos: APIs REST com Express.js, Conexão com MongoDB (Mongoose), Relacionamentos (Referências / ObjectIds e `populate`), Múltiplos Schemas e CRUDs, Validações de Schema, Lógicas de Negócio complexas, Query Params (filtros) e Variáveis de Ambiente.

1. Contexto

A startup **Imóvel Lions** precisa de um sistema completo para gerenciar imóveis de aluguel por temporada, as reservas dos clientes e a experiência dos usuários por meio de avaliações (reviews).

Para este projeto, você precisará modelar três recursos que se relacionam no banco de dados MongoDB: o **Imóvel**, a **Reserva** e a **Avaliação**. Como ainda não utilizam o padrão MVC completo com controllers separados, a estrutura deve seguir o modelo modular simplificado aprendido em aula (`db.js` , `server.js` , `models/` e `routes/`).

2. Configuração Inicial e Estrutura de Pastas

Organize sua aplicação com a seguinte estrutura de arquivos:

```
Imovellions/  
├── models/  
│   ├── imovel.js  
│   ├── reserva.js  
│   └── avaliacao.js  
├── routes/  
│   ├── imovel.js  
│   ├── reserva.js  
│   └── avaliacao.js  
├── db.js  
├── server.js  
├── .env  
└── package.json
```

Configure o arquivo `.env` com a sua URI de conexão do MongoDB Atlas (`MONGO_URI`) e a porta do servidor (`PORT=3000`).

2.1 Modelos de Dados (Schemas)

Defina os Schemas do Mongoose detalhados a seguir:

Modelo: Imóvel (`models/imovel.js`)

- `titulo` : Tipo `String` , obrigatório.
- `descricao` : Tipo `String` , obrigatório.
- `localizacao` : Tipo `String` , obrigatório (ex: `"Ubatuba - SP"`).
- `precoNoite` : Tipo `Number` , obrigatório (valor base da diária).
- `capacidadeMaxima` : Tipo `Number` , obrigatório.
- `disponivel` : Tipo `Boolean` , com valor padrão `true` .

Modelo: Reserva (`models/reserva.js`)

- `imovelId` : Tipo `mongoose.Schema.Types.ObjectId` com referência (`ref`) ao modelo `Imovel` , obrigatório.
- `nomeHospede` : Tipo `String` , obrigatório.
- `emailHospede` : Tipo `String` , obrigatório.
- `dataEntrada` (Check-in): Tipo `Date` , obrigatória.
- `dataSaida` (Check-out): Tipo `Date` , obrigatória.
- `hospedes` : Array de Objetos, obrigatório, onde cada objeto deve conter:
 - `nome` : Tipo `String`
 - `idade` : Tipo `Number`
- `cupomDesconto` : Tipo `String` , opcional.
- `valorTotal` : Tipo `Number` (calculado automaticamente pela API).
- `status` : Tipo `String` , padrão `"Pendente"` (deve aceitar apenas: `Pendente` , `Confirmada` ou `Cancelada`).

Modelo: Avaliação (`models/avaliacao.js`)

- `imovelId` : Tipo `mongoose.Schema.Types.ObjectId` com referência (`ref`) ao modelo `Imovel` , obrigatório.
- `nomeUsuario` : Tipo `String` , obrigatório.
- `nota` : Tipo `Number` , obrigatório (deve aceitar apenas valores de `1` a `5`).
- `comentario` : Tipo `String` , obrigatório (mínimo de 10 caracteres).
- `dataCriacao` : Tipo `Date` , padrão `Date.now` .

3. Requisitos Obrigatórios e Regras de Negócio (Nível Avançado)

Você deverá implementar rotas para gerenciar Imóveis (`/imoveis`), Reservas (`/reservas`) e Avaliações (`/avaliacoes`).

3.1 CRUD de Imóveis

- **Cadastrar Imóvel (POST `/imoveis`)**: Cria um novo imóvel. Retorne status `201`.
- **Listar Imóveis (GET `/imoveis`)**: Retorna todos os imóveis cadastrados.
- **Buscar Imóveis por Localização (GET `/imoveis/busca`)**: Deve aceitar o Query Param `localizacao`. Retorna apenas os imóveis que contêm o termo pesquisado na localização (ex: buscar por `"uba"` deve retornar imóveis em `"Ubatuba - SP"`).

3.2 CRUD de Reservas (Lógica de Negócio)

- **Criar Reserva (POST `/reservas`)**: Recebe no corpo: `imovelId`, `nomeHospede`, `emailHospede`, `dataEntrada`, `dataSaida`, `hospedes` (array) e `cupomDesconto` (opcional).

Regras de Negócio (Lógica em JavaScript):

1. **Validação de Existência**: A API deve verificar se o imóvel informado realmente existe no banco de dados. Se não existir, retorne status `404` com erro.
2. **Validação de Capacidade**: Verifique se a quantidade total de hóspedes (tamanho do array `hospedes`) não ultrapassa a `capacidadeMaxima` do imóvel selecionado. Se ultrapassar, retorne status `400` com erro.
3. **Cálculo de Diárias por Dia da Semana**: O preço base do imóvel (`precoNoite`) é válido apenas para dias de semana (segunda a quinta-feira). Para cada noite reservada (do dia de entrada ao dia anterior ao de saída):
 - Se a noite for de **sexta-feira, sábado ou domingo**, a diária daquela noite sofre um **acréscimo de 20%** sobre o valor base do imóvel.
 - Se for de **segunda a quinta-feira**, o valor cobrado é o preço base normal.
4. **Taxa de Hóspede Adicional por Idade**: O valor da diária básica cobre até **2 hóspedes** (independente da idade). A partir do **3º hóspede** (índice 2 do array em diante), deve ser cobrada uma taxa adicional por diária:
 - Hóspede adicional com **12 anos ou mais**: acréscimo de **R\$ 50** por noite.
 - Hóspede adicional com **menos de 12 anos**: acréscimo de **R\$ 25** por noite.
5. **Desconto de Longa Estadia**: Se a estadia tiver **5 noites ou mais**, aplique um **desconto de 10%** sobre a soma acumulada de todas as diárias (incluindo as taxas de hóspedes adicionais).
6. **Cupom de Desconto**: Se o cliente enviar o cupom `"LIONS10"`, conceda um **desconto adicional de 10%** sobre o valor final após todas as contas anteriores.
7. Se todas as regras passarem, salve a reserva no banco de dados com o `valorTotal` calculado e retorne o documento com status `201`.

- **Listar Reservas (GET /reservas)**: Retorna todas as reservas utilizando `.populate("imovelId")`.
- **Alterar Status da Reserva (PATCH /reservas/:id/status)**: Permite atualizar apenas o campo `status` da reserva. Retorne status `200`.
- **Relatório Financeiro e Estatísticas (GET /reservas/relatorio)**:
 - Busque todas as reservas com `status: "Confirmada"`.
 - Utilizando loops simples em JavaScript (sem agregação de banco de dados), calcule e retorne:
 - `faturamentoTotal`: a soma do `valorTotal` de todas as reservas confirmadas.
 - `totalNoites`: a soma total de noites de todas as estadias confirmadas.
 - `mediaHospedes`: a média de hóspedes por reserva confirmada.

3.3 CRUD de Avaliações (Relacionamento cruzado)

- **Criar Avaliação (POST /avaliacoes)**: Recebe no corpo: `imovelId`, `nomeUsuario`, `nota` (de 1 a 5) e `comentario`.

Regras de Negócio:

1. **Validação de Imóvel**: Verifique se o imóvel avaliado realmente existe no banco de dados. Caso contrário, retorne status `404`.
 2. **Regra de Hóspede Real**: Um usuário só pode avaliar um imóvel se ele possuir pelo menos uma reserva cadastrada para aquele imóvel com o status `"Confirmada"`.
 - *Dica*: Busque no banco de reservas se existe algum documento com o `imovelId` e `nomeHospede` correspondentes e com o `status: "Confirmada"`. Se não existir, retorne status `400` informando que apenas hóspedes reais podem avaliar o imóvel.
 3. Se passar, salve a avaliação e retorne o documento criado com status `201`.
- **Listar Avaliações de um Imóvel (GET /avaliacoes/imovel/:imovelId)**:
 - Retorna todas as avaliações feitas para o imóvel especificado.
 - **Regra de Negócio (Média Geral)**: A resposta deve retornar a lista de avaliações e a média aritmética das notas (calculada em JavaScript).
 - **Excluir Avaliação (DELETE /avaliacoes/:id)**:
 - **Regra de Segurança**: O usuário deve enviar seu `nomeUsuario` no corpo da requisição. O backend deve verificar se a avaliação de fato pertence a esse usuário. Se pertencer, remova a avaliação do banco; caso contrário, retorne status `403` (Proibido) com uma mensagem de erro.

4. Testes Esperados

Realize testes na sua API simulando o seguinte fluxo de validação:

1. Cadastre um imóvel com capacidade máxima de 4 pessoas e **precoNoite** de R\$ 100.
 2. Crie uma reserva para 3 hóspedes. Verifique se a diária e a taxa do hóspede adicional foram calculadas e grave a reserva.
 3. Tente criar uma avaliação para o imóvel utilizando um usuário qualquer (deve falhar com erro 400, pois esse usuário ainda não tem reserva confirmada).
 4. Mude o status da reserva do passo 2 para **"Confirmada"**.
 5. Tente criar a avaliação novamente usando o mesmo nome do hóspede cadastrado na reserva (deve salvar com sucesso).
 6. Liste as avaliações do imóvel e verifique se a média foi calculada corretamente.
 7. Tente excluir a avaliação enviando um nome de usuário diferente no corpo (deve falhar com erro 403).
 8. Exclua a avaliação enviando o nome de usuário correto.
-

5. Dicas para a Implementação

- Na rota de criar avaliação, verifique se o usuário já se hospedou:

```
const reservaAtiva = await Reserva.findOne({
  imovelId,
  nomeHospede: nomeUsuario,
  status: "Confirmada"
});
if (!reservaAtiva) {
  return res.status(400).json({ message: "Apenas hóspedes com reservas confirmadas podem avaliar." });
}
```

- No modelo de avaliações, valide a nota com **min: 1** e **max: 5** no próprio Schema do Mongoose.
 - Use o **try/catch** para capturar as validações disparadas pelo Schema do Mongoose na hora de salvar.
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de Mongoose e MongoDB - Módulo 08